

Лабораторная работа.

Агентный искусственный интеллект

Список задач

- Подготовить воспроизводимую среду для запуска открытой языковой модели или совместимого интерфейса вызова модели.
- Реализовать два режима решения одной и той же задачи: базовый неагентный режим и агентный режим с инструментами.
- Подготовить набор тестовых запросов и эталонную структуру ответа для проведения экспериментов.
- Реализовать журнал трассировки (trace) с фиксацией шагов, состояний, вызовов инструментов и причин завершения.
- Разработать процедуру оценки, включающую метрики качества результата, качества процесса и эксплуатационных характеристик.
- Провести серию контролируемых экспериментов, сравнить варианты архитектуры и сделать выводы по наблюдаемым различиям.

Что включить в отчет

- Описание используемой модели, инструментов, структуры состояния и логики переходов между шагами.
- Описание тестового набора запросов, схемы экспериментов и исследуемых факторов.
- Таблицы с результатами сравнения baseline и agent-режима, а также сравнения agent без evaluator и agent с evaluator.
- Графики или диаграммы по времени выполнения, числу шагов, полноте ответа и итоговой экспертной оценке.
- Разбор типичных ошибок, неудачных траекторий, причин деградации и границ применимости реализованного решения.

Основы агентного искусственного интеллекта

1. Введение в агентные системы

Агентная система отличается от обычного LLM-приложения тем, что решает задачу не в один проход, а как последовательность действий: получает цель, хранит состояние, выбирает следующий шаг, вызывает инструменты, анализирует промежуточный результат и завершает выполнение по явному критерию остановки.

Для лабораторной работы используется прикладной сценарий научно-информационного поиска. Система должна не просто ответить на вопрос, а собрать обзор по теме, найти релевантные публикации, сопоставить несколько источников, сформировать структурированное резюме и показать, как именно был получен результат.

2. Почему в работе нужен именно агентный подход

- Результат зависит от нескольких шагов: поиск общей справочной информации, поиск научных публикаций, фильтрация результатов, извлечение ключевых сведений, формирование итоговой сводки.
- Маршрут выполнения зависит от промежуточных результатов. Если публикаций найдено мало, требуется повторный поиск с изменением формулировки; если найденные источники слишком общие, требуется дополнительная фильтрация.
- Задача требует работы с инструментами: поиском по открытым API, загрузкой метаданных публикаций, обработкой аннотаций и вычислением агрегированных показателей.
- Качество решения определяется не только финальным текстом, но и качеством траектории: количеством шагов, корректностью вызовов инструментов, полнотой собранных полей и устойчивостью к ошибкам.

3. Архитектурная идея лабораторной работы

Работа строится вокруг сравнения двух режимов. Первый режим – baseline, в котором модель получает тему и краткий контекст из поиска, после чего сразу формирует итоговый ответ. Второй режим – agent, где система последовательно вызывает инструменты, хранит состояние, при необходимости переформулирует запросы и дополнительно проверяет итоговый ответ через evaluator.

Определения терминов

1. Agent – система, выполняющая задачу как последовательность шагов с использованием состояния, инструментов и критерия остановки.
2. Tool – функция или внешний интерфейс, к которому обращается агент: HTTP API, поиск, извлечение абстракта, локальная обработка текста.
3. State – структура данных, в которой хранится цель, история действий, найденные источники, промежуточные выводы и статус выполнения.
4. Trace – подробная запись траектории: шаг, аргументы вызова, ответ инструмента, обновление состояния, причина следующего перехода.
5. Planner – логика выбора следующего действия на основании текущего состояния.
6. Evaluator – компонент, оценивающий итоговый результат по рубрике и проверяющий полноту обязательных полей.

7. Task success – успешность выполнения задачи в целом: получен ли приемлемый структурированный обзор по заданной теме.
8. Groundedness – степень опоры ответа на реально найденные и зафиксированные источники.
9. Coverage of required fields – наличие в итоговом ответе всех обязательных элементов: определение, 3–5 источников, методы, применения, ограничения.
10. Latency – общее время выполнения одного запроса от старта до выдачи результата.
11. Escalation / fallback – переход в безопасный режим, когда агент не может продолжать выполнение допустимым образом.

Постановка задачи и структура эксперимента

1. Общая постановка

Необходимо реализовать систему, которая по формулировке темы строит краткий научно-аналитический обзор в следующей структуре: определение темы, основные подходы, 3–5 ключевых работ, возможные приложения, ограничения, список использованных источников.

Для каждой темы требуется получить результат в двух конфигурациях: baseline и agent. Дополнительно для agent требуется исследовать вариант без evaluator и вариант с evaluator.

2. Источники и ограничения

- Рекомендуется использовать только открытые и воспроизводимые источники: OpenAlex API, arXiv API, Crossref API, Semantic Scholar open endpoints, Wikipedia API, локальные открытые статьи или аннотации.
- Если используется локальная модель через Ollama, в отчете фиксируются название модели, квантование и аппаратная среда. Если используется удаленный открытый endpoint, фиксируются версия и условия вызова.

3. Набор запросов для экспериментов

Экспериментальный набор должен содержать не менее восьми тем. Рекомендуемый набор, но можно выбрать свои темы:

- Agentic AI for customer support
- Graph RAG for enterprise knowledge systems
- LLM evaluation and process-aware metrics
- Tool-using language models in scientific search
- Retrieval-augmented generation in medicine
- Planning and reflection in LLM agents
- Human-in-the-loop AI systems
- Knowledge graphs for procedural reasoning

4. Сравниваемые конфигурации

Конфигурация	Описание	Инструменты	Ожидаемая роль
Baseline	Одношаговый ответ по теме на основе краткого поискового контекста	Поиск + LLM	Точка сравнения
Agent	Итеративное выполнение с состоянием и инструментами	Поиск, публикации, извлечение абстрактов	Основная система
Agent + evaluator	Agent с дополнительной проверкой полноты и качества	Инструменты агента + evaluator	Проверка эффекта контроля качества

Реализация инструментов и состояния

1. Базовые библиотеки

```
import json
import time
```

```
import requests
from dataclasses import dataclass, field
from typing import List, Dict, Any
```

2. Структура состояния агента

```
@dataclass
class AgentState:
    topic: str
    objective: str
    step_id: int = 0
    history: List[Dict[str, Any]] = field(default_factory=list)
    sources: List[Dict[str, Any]] = field(default_factory=list)
    notes: List[str] = field(default_factory=list)
    final_answer: str = ""
    status: str = "running"
    stop_reason: str = ""
```

Состояние должно обновляться после каждого шага. Минимально фиксируются идентификатор шага, действие, аргументы, краткий результат, изменение числа найденных источников и причина перехода к следующему шагу.

3. Инструмент поиска публикаций через OpenAlex

```
def search_openalex(query: str, per_page: int = 5) -> List[dict]:
    url = "https://api.openalex.org/works"
    params = {
        "search": query,
        "per-page": per_page,
        "select": "id,display_name,publication_year,abstract_inverted_index,authorships"
    }
    response = requests.get(url, params=params, timeout=30)
    response.raise_for_status()
    return response.json()["results"]
```

4. Преобразование abstract_inverted_index в текст

```
def invert_abstract(inv_idx: dict) -> str:
    if not inv_idx:
        return ""
    words = []
    for token, positions in inv_idx.items():
        for pos in positions:
            words.append((pos, token))
    words.sort(key=lambda x: x[0])
    return " ".join(token for _, token in words)
```

5. Инструмент общего справочного поиска через Wikipedia API

```
def search_wikipedia(query: str) -> str:
    url = "https://en.wikipedia.org/api/rest_v1/page/summary/" + query.replace(" ", "_")
    response = requests.get(url, timeout=30)
    if response.status_code != 200:
        return ""
    data = response.json()
    return data.get("extract", "")
```

Логика baseline и логика agent

1. Baseline-режим

Baseline должен выполнять только три шага: получить тему, извлечь краткий общий контекст, сформировать итоговый ответ. Пример минимальной реализации:

```
def run_baseline(topic: str, llm_call):
    wiki_context = search_wikipedia(topic)
    prompt = f'''
    Сформируй краткий обзор по теме: {topic}.
    Используй следующий контекст:
    {wiki_context}

    Структура ответа:
    1) определение темы
    2) основные подходы
    3) 3-5 ключевых работ
    4) применения
    5) ограничения
    '''
    return llm_call(prompt)
```

2. Agent-режим

Agent должен как минимум: собрать общий контекст, найти публикации, отобрать релевантные результаты, извлечь заметки, сформировать итоговый ответ и сохранить trace.

```
def log_step(state: AgentState, action: str, payload: dict, result: str):
    state.history.append({
        "step_id": state.step_id,
        "action": action,
        "payload": payload,
        "result": result[:300]
    })
    state.step_id += 1

def run_agent(topic: str, llm_call, max_steps: int = 6) -> AgentState:
    state = AgentState(topic=topic, objective="Подготовить структурированный обзор темы")
    wiki_context = search_wikipedia(topic)
    log_step(state, "search_wikipedia", {"topic": topic}, wiki_context)

    papers = search_openalex(topic, per_page=8)
    state.sources = papers
    log_step(state, "search_openalex", {"query": topic, "per_page": 8},
    f"found={len(papers)}")

    notes = []
    for paper in papers[:5]:
        abstract = invert_abstract(paper.get("abstract_inverted_index"))
        notes.append({
            "title": paper.get("display_name", ""),
            "year": paper.get("publication_year", ""),
            "abstract": abstract[:1200]
        })
    state.notes = notes
    log_step(state, "extract_notes", {"n_sources": len(notes)}, "notes_prepared")

    prompt = f'''
    Подготовь научно-аналитический обзор по теме: {topic}

    Общий контекст:
    {wiki_context}'''
```

```

Источники:
{json.dumps(notes, ensure_ascii=False)}

Обязательная структура:
- определение
- основные подходы
- 3-5 ключевых работ
- применения
- ограничения
- использованные источники
...

state.final_answer = llm_call(prompt)
state.status = "finished"
state.stop_reason = "final_answer_generated"
log_step(state, "generate_final_answer", {"topic": topic}, state.final_answer)
return state

```

3. Evaluator

Evaluator должен быть отделен от основного цикла и использовать фиксированную рубрику. Пример ответа evaluator – JSON-структура с оценками по критериям.

```

EVAL_PROMPT = '''
Оцени ответ по шкале от 0 до 5 по критериям:
1. correctness
2. groundedness
3. completeness
4. coverage_of_required_fields
5. source_consistency

Верни только JSON:
{
  "correctness": int,
  "groundedness": int,
  "completeness": int,
  "coverage_of_required_fields": int,
  "source_consistency": int,
  "comment": "..."
}
'''

def evaluate_answer(answer: str, notes: list, llm_call):
    prompt = EVAL_PROMPT + "\nОтвет:\n" + answer + "\n\nИсточники:\n" + json.dumps(notes,
    ensure_ascii=False)
    raw = llm_call(prompt)
    return json.loads(raw)

```

4. Запись результатов эксперимента

```

def save_trace(state: AgentState, path: str):
    with open(path, "w", encoding="utf-8") as f:
        json.dump({
            "topic": state.topic,
            "history": state.history,
            "n_sources": len(state.sources),
            "status": state.status,
            "stop_reason": state.stop_reason
        }, f, ensure_ascii=False, indent=2)

```

Экспериментальная оценка

1. Почему основной акцент должен быть сделан на оценке

В лабораторной работе ценность определяется не самим фактом генерации текста, а способностью провести сравнимые эксперименты. Поэтому оценка должна быть организована как отдельный воспроизводимый контур, а не как краткая иллюстрация в конце.

2. Группы метрик

Группа	Метрика	Смысл	Как измеряется
Качество результата	correctness / task success	Насколько итоговый обзор пригоден для решения задачи	Evaluator или экспертная рубрика 0–5
Качество результата	factuality и groundedness	Насколько выводы опираются на найденные источники	Сопоставление ответа с notes и ручная проверка
Качество результата	completeness и coverage of required fields	Присутствуют ли все обязательные поля	Автоматическая проверка по шаблону + evaluator
Качество процесса	число шагов	Насколько экономно агент достигает результата	len(trace)
Качество процесса	доля лишних действий	Сколько шагов не дали новой полезной информации	Ручная маркировка или правило
Эксплуатационные	latency	Полное время выполнения одного запроса	time.time()
Эксплуатационные	ошибки инструментов	Стабильность вызовов API и обработчиков	Подсчет исключений и fallback
Сводная	rubric score	Интегральная экспертная оценка	Среднее по критериям evaluator

3. Пример таблица для регистрации результатов

Тема	Конфигурация	Correctness	Groundedness	Completeness	Шаги	Latency, с	Ошибки tools	Rubric

Проведение экспериментов

1. План экспериментов

- Эксперимент 1. Baseline на восьми темах.
- Эксперимент 2. Agent без evaluator на тех же восьми темах.
- Эксперимент 3. Agent с evaluator на тех же восьми темах.
- Эксперимент 4. Сравнение agent при разном числе источников: top-3, top-5 и top-8.
- Эксперимент 5. Сравнение agent при ограничении по максимальному числу шагов: 4, 6 и 8.

2. Шаблон цикла эксперимента

```
def run_experiment(topics, mode_name, runner, llm_call):
    records = []
    for topic in topics:
        start = time.time()
        result = runner(topic, llm_call)
        latency = time.time() - start

        if isinstance(result, AgentState):
            answer = result.final_answer
            notes = result.notes
            n_steps = len(result.history)
        else:
            answer = result
            notes = []
            n_steps = 1

        eval_json = evaluate_answer(answer, notes, llm_call)
        records.append({
            "topic": topic,
            "mode": mode_name,
            "correctness": eval_json["correctness"],
            "groundedness": eval_json["groundedness"],
            "completeness": eval_json["completeness"],
            "coverage": eval_json["coverage_of_required_fields"],
            "rubric": sum([
                eval_json["correctness"],
                eval_json["groundedness"],
                eval_json["completeness"],
                eval_json["coverage_of_required_fields"],
                eval_json["source_consistency"]
            ]) / 5.0,
            "n_steps": n_steps,
            "latency": latency
        })
    return records
```

3. Агрегация результатов

```
import pandas as pd

all_records = []
all_records += run_experiment(topics, "baseline", run_baseline, llm_call)
all_records += run_experiment(topics, "agent", run_agent, llm_call)

df = pd.DataFrame(all_records)
summary = df.groupby("mode")[["correctness", "groundedness", "completeness", "rubric",
    "n_steps", "latency"]].mean()
print(summary)
```

4. Визуализация

```
import matplotlib.pyplot as plt

summary[["correctness", "groundedness", "completeness", "rubric"]].plot(kind="bar",
figsize=(10, 5))
plt.title("Сравнение качества результатов")
plt.ylabel("Средний балл")
plt.tight_layout()
plt.show()

summary[["n_steps", "latency"]].plot(kind="bar", figsize=(8, 4))
plt.title("Сравнение процесса выполнения")
plt.tight_layout()
plt.show()
```

5. Анализ неудачных кейсов

Для не менее чем трех тем необходимо вручную разобрать trace и указать, на каком шаге возникла основная потеря качества. Рекомендуется классифицировать ошибки по типам: слабый поиск, нерелевантный источник, потеря полноты, неверная интерпретация аннотации, отсутствие связи между выводом и источниками, лишние шаги.

Требования к отчету и итоговому выводу

- В отчете должна присутствовать явная постановка сравнения между baseline и agent, а не только описание одного решения.
- Таблицы и графики должны содержать результаты как минимум по трем конфигурациям: baseline, agent, agent + evaluator.
- Необходимо отдельно интерпретировать различия между качеством результата и качеством процесса. Улучшение rubric score при резком росте latency и числа шагов должно быть осмысленно прокомментировано.
- Итоговый вывод должен содержать не только констатацию лучшего варианта, но и объяснение, почему именно он оказался лучше: за счет полноты, лучшего grounding, меньшего числа лишних шагов или большей устойчивости к ошибкам поиска.